BDAP Assignment 1

Online Spam Filters

1. Accuracy

The results of the accuracy of the different algorithms are given in the tables below. The accuracy shown in the tables is the accuracy of the algorithms on the last data examples. The different algorithms are the Naive Bayes and the perceptron classifier, both implemented by using feature hashing and count-min sketch to deal with the large amount of data.
For the Naive Bayes classifier with feature hashing, we change the amount of buckets and the size of the ngrams. For the count-min sketch variant of Naive Bayes, we also change the amount of buckets but we keep the ngram size equal to 3 because the accuracy is best when using this setting. We use 5 different hash functions in the experiment. The threshold of the algorithm is set to 0 in all experiments for a reason that has to do with the way of implementing the algorithm. This will be explained in section 4 of this report.

Table 1: Accuracy Naive Bayes with feature hashing

Log_num_buckets	ngram size = 1	ngram size = 3	ngram size = 6
10	45.9	39.6	38.9
15	70.8	48.0	42.4
20	79.1	74.9	61.3
25	79.6	**88.9**	84.9
30	79.6	**89.7**	86.6

Table 2: Accuracy Naive Bayes with count-min sketch

Log_num_buckets	ngram size = 3
10	39.5
15	47.9
20	77.0
25	**89.7**

For the perceptron classifier with feature hashing we change the amount of buckets and the size of the ngrams. Table 2 also shows the accuracy if we change the learning rate. For the count-min sketch variant of the perceptron we again change the amount of buckets and keep the ngram size equal to 3. In this experiment, 5 different hash functions are used.

Table 3: Accuracy Perceptron with feature hashing

Log_num_buckets	ngram size = 1	ngram size = 3	ngram size = 6
5	68.7	63.9	61.0
10	89.4	84.9	80.8
15	**91.9**	**94.6**	**92.3**

Table 4: Accuracy Perceptron and changing learning rate

Learning rate	ngram size = 3
0.01	**94.6**
0.1	61.2
0.2	56.7

Table 5: Accuracy Perceptron with count-min sketch

Log_num_buckets	ngram size = 3
5	69.2
10	91.3
15	**95.3**

1.1. Naive Bayes classifiers

If we look at the results of Naive Bayes with feature hashing in table 1, we can see the effect of changing the number of buckets on the accuracy of the algorithm. If we increase the number of buckets, the accuracy increases. The accuracy is lower for a smaller amount of buckets because the amount of collisions is higher than with a larger amount of buckets. Collisions occur when multiple ngrams map to the same bucket. If the amount of collisions is too high, the difference between important and unimportant features for spam and ham mails will not have a big enough influence to make decisive definition in the classification. We can see that the best accuracy is when the log_num_buckets is 25 or 30. The disadvantage of the large amounts of buckets is that the algorithm is slower and takes more space in memory. The algorithm is slower because the buckets needed during a prediction are farther apart from each other in memory. We can also see that the accuracy is highest when the size of the ngrams is equal to 3, but the difference with ngram size 6 is not big. However, the performance of the algorithm is better with ngram size 3 because less ngrams are used with each email. This makes ngram size 3 the optimal choice.

For the count-min sketch variant of Naive Bayes (table 2) we can also ask ourselves the question about what the effect would be of changing the number of buckets. We see slightly better accuracies in comparison with feature hashing and the same effect of changing the number of buckets. Changing the amount of different hash functions has no big influence. The increase in the amount of hash functions does have an effect on the speed of the algorithm. More hash functions mean that the algorithm needs to do more lookups in a bigger vector of buckets and do more computations.

1.2. Perceptron classifiers

For the perceptron classifier with feature hashing the results are show in table 3. We again see that a higher amount of buckets leads to a better accuracy. The reason is the same, namely that a small amount of buckets lead to more collisions.
We can also look at the effect of changing the learning rate. The results in table 4 show that the bigger the learning rate, the worse the accuracy. The bigger learning rate changes the weights too much each iteration which prevents converging too the optimal weights.

For the count-min sketch variant of the Perceptron classifier (table 5), we see the same effects as for the count-min sketch implementation of the Naive Bayes classifier.

1.3. Learning curves

The learning curves of the accuracy of the algorithms are shown in the figures at the end of this report. The learning curves are created with the whole dataset for all the algorithms. We see that the learning curves increase when more emails are processed, thus the accuracy gets better each iteration.
Figure 5 and 6 show a zoomed in view of the first 5000 examples of the learning curves for Naive Bayes with feature hashing and the Perceptron with count-min sketch. We can see that the accuracy fluctuate a bit for the first examples. This happens because a lot of buckets are still empty or a lot of weights are still 0 and thus the algorithm is still uncertain about the classification.

2. Other evaluation metrics

2.1. Precision and Recall

Another evaluation metric that can be used to evaluate classifiers is precision. Precision indicates the percentage of emails classified as spam that are true spam emails. This is an important metric for a spam classifier because falsely classified mails as spam mails should be avoided. Precision is also more important than recall, which indicates how much of the actual spam mails are classified as spam. Of course, a good recall is makes the algorithm even better. We can see in table 6 that the precision for all the algorithms is more than acceptable. The f1-score is an evaluation metric that measures the tradeoff between precision and recall. If one of the two is low, the recall will also be low. So we can use the f1-score to make sure that we have a classifier with high enough precision, as this is the most important, without losing too much on the recall side. The results for the f1-score show that the perceptron classifiers perform better than the Naive Bayes.

Table 6: Precision and F1score different classifiers

	NB feature hashing	NB count-min sketch	Perceptron feature hashing	Perceptron count-min sketch
Precision	98.5	98.4	94.5	94.2
F1score	87.6	88.6	**95.6**	**95.4**

2.2. Log-loss

The last evaluation metric that is used to evaluate the algorithms is log-loss. Log-loss indicates how close the prediction probability is to the actual value. We cannot use this metric to evaluate the Naive Bayes classifiers because the classification is based on the difference between the probability of the email being spam and ham. Log-loss can be used for the perceptron classifiers because the output of the classifier is a probability between -1 and 1 (converted to the range from 0 to 1). We can use log-loss to decide which parameter settings such as the amount of buckets is preferable. The classification with a lower log-loss value is better than one with a higher log-loss value. We expect that the log-loss value for a larger amount of buckets will be lower because there will be less collision in the buckets, which will lead to a more certain prediction. The results in table 7 indeed show our expectations. Log-loss always needs to be compared to the baseline model. If the log-loss of the classifier is not better than the baseline model, the classifier is not useful. The baseline model for this dataset is just predicting that an email is spam with a probability of 0.61 for every email. The reason for this is that 61 percent of the mails in the dataset are spam. The log-loss of the baseline model is 0.82, which means that our algorithm performs better than the baseline model.

Table 7: Log-loss for Perceptron with feature hashing

Log_num_buckets	Log-loss
5	0.665
10	0.388
15	**0.256**

3. Efficiency

We can ask ourselves what the impact is of the amount of buckets or the number of hash functions on the speed of the algorithm. If we run the algorithms with a larger amount of buckets, the efficiency is lower. The reason for this is that the buckets needed for the classification for a specific email are spaced farther apart in memory. Looking up these buckets will be a little bit slower. For the perceptron classifier, all the weights in the buckets are used in the computation of the probability for spam mails. If the amount of buckets is larger, it will of course take more time to use these buckets in the calculation. We can see the same effect in the count-min sketch implementations. If we use more hash functions, the algorithm will be slower. To improve the performance, one vector with the buckets from the different hash functions behind each other is used instead of using a 2D-vector. This will make sure the buckets are all behind each other in memory which is not necessarily the case with a 2D-vector.

4. Implementation details

In this final section, I will explain a couple of important implementation details.

The first detail is how the classification is determined in the Naive Bayes algorithms. The formula that is used to calculate the prediction, is the formula on slide 10 of the lecture slides on scaling learning.

$$\log\frac{prob(email\ is\ spam)}{prob(email\ is\ ham)} = \log\frac{prob(spam)}{prob(ham)} + \log\frac{prob(X1|S)}{prob(X1|H)} + \cdots + \log\frac{prob(Xn|S)}{prob(Xn|H)}$$

This formula gives the difference between the probability of the email being spam and the probability of the email being ham. The result of this formula will be positive if the email is classified as spam and negative if the email is classified as ham, so the threshold that should be used is 0. The result will be positive if the probability on the email being spam is higher than the probability of the email being ham. We can use this formula because we only care about the classification, so about the result being positive or negative.

The count-min sketch implementation of Naive Bayes uses multiple hash function to hash the ngrams. The counts for the different hash values are stored in one big vector behind each other. When a prediction is made, the minimum count for the ngram from the different hash values is taken to compute the prediction. We can validate our implementation of count-min sketch algorithm by taken the amount of hash functions equal to 1. This is then the same algorithm as the feature hashing implementation. Thus, the result is the same as the result from the feature hashing implementation.

The perceptron classifier combines each weight with the corresponding input for that weight to calculate the output. For this reason, I created a function that computes the input feature vector. This vector has the same amount of buckets as there are weights. The buckets are filled by computing the hash value of each ngram in the email and increasing the count of the correct bucket. Because some ngrams occur multiple times or different ngrams map to the same bucket, the count of a bucket can be bigger than 1. To have an input value between 0 and 1, the input feature vector is normalized such that the biggest value is mapped to 1. This will make sure that the ngram that occurs most often will have the highest contribution in the classification. It is also possible to just add the weight of an ngram each time the ngram occurs. But this can influence the result too much if one weight has a large value and is added multiple times. I noticed that this could lead to a weight vector where all weights eventually grow to -1 or 1 which leads to a poor prediction.

Multiple hash functions are used in the count-min sketch algorithm. This gives multiple weight vectors which are stored in one big vector behind each other. For each hash function, also an input feature vector is computed for the emails. Each ngram is hashed with multiple hash functions and the different weights are updated based on the error, learning rate and the corresponding input value. The median weight instead of the minimum is used for each ngram in an email during the prediction step.

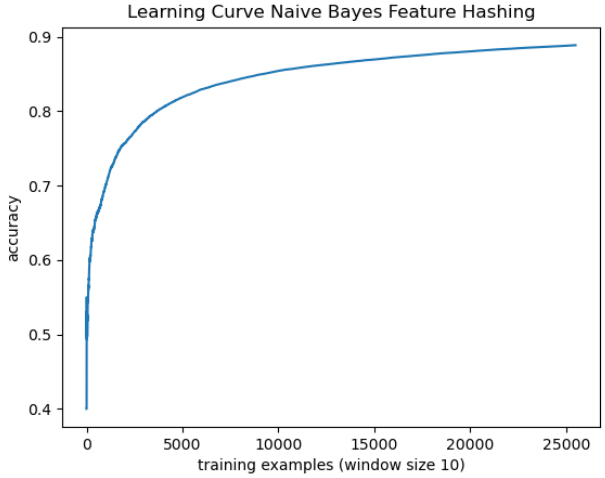


Figure 1: Learning curve accuracy Naive Bayes with feature hashing

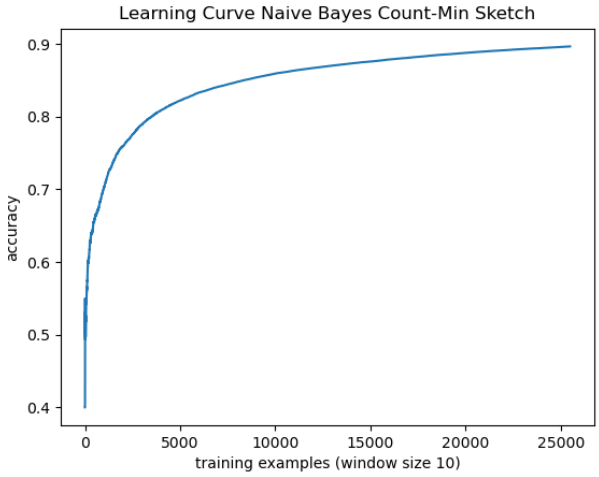


Figure 2: Learning curve accuracy Naive Bayes with count-min sketch

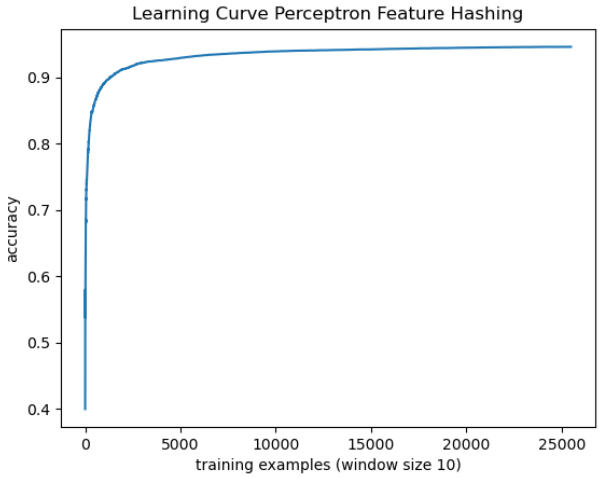


Figure 3: Learning curve accuracy Perceptron with feature hashing

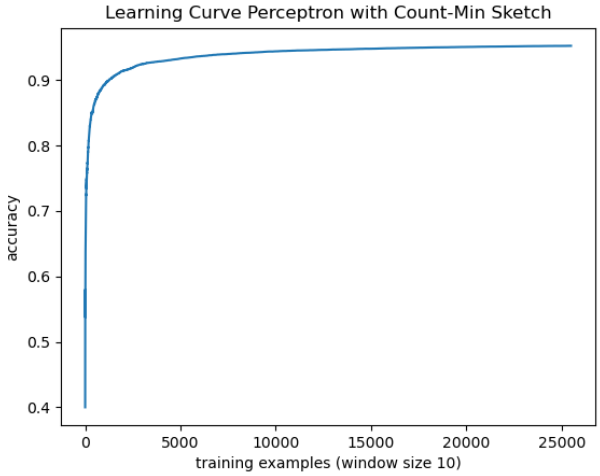


Figure 4: Learning curve accuracy Perceptron with count-min sketch

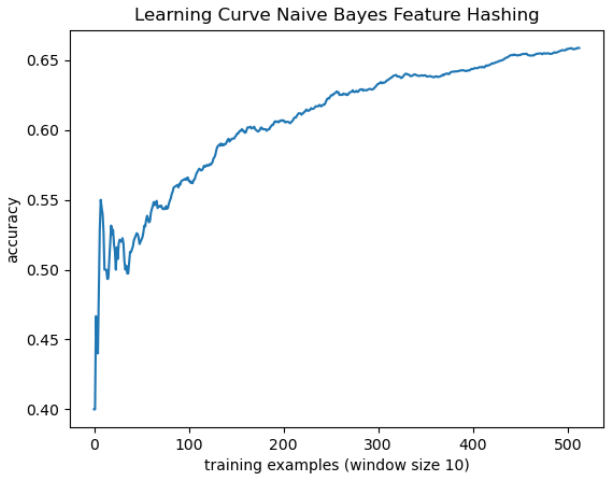


Figure 5: Learning curve accuracy Naive Bayes with feature hashing zoomed in

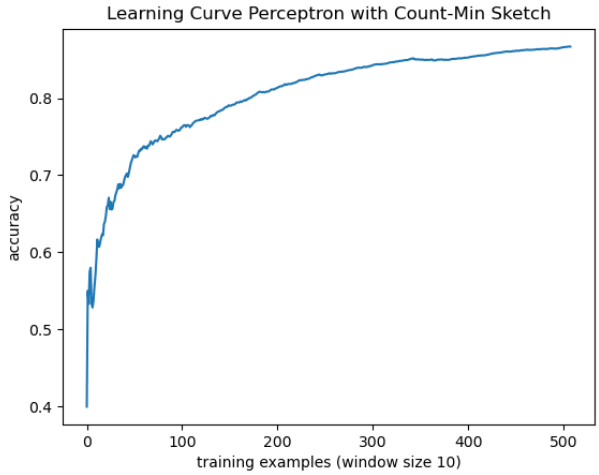


Figure 6: Learning curve accuracy Perceptron with count-min sketch zoomed in